ERC-20 토큰 학습 가이드

TokenZswap 프로젝트의 SunWoo42(SW42) · Ra2techB(R2B)를
한 줄씩, 단위 계산까지 따라가며 이해하기

대상 파일: `code/src/SunWoo42.sol`, `code/src/tokenB.sol`

함께 보는 라이브러리: OpenZeppelin Contracts 5.7.0 (`ERC20`, `ERC20Permit`, `Ownable`)

Solidity 0.8.28 · Foundry

이 문서의 숫자 예시는 실제 Foundry 테스트와 anvil 실행으로 확인한 값입니다.

목차

1. ERC-20이란: 장부 3개와 함수 6개
2. 단위: wei, decimals, 10^{18} (계산 연습 포함)
3. 파일 첫 부분: SPDX, pragma, import, NatSpec
4. 상속: `is ERC20, ERC20Permit, Ownable`
5. 상태 변수: constant, mapping, event
6. constructor: 토큰이 태어나는 순간
7. OpenZeppelin ERC20 내부 들여다보기
8. faucet(): 누구나 1일 1회 1000개
9. mint()와 onlyOwner, Ownable
10. ERC20Permit: 서명으로 하는 approve
11. 토큰의 일생: 장부 변화 추적 (숫자 시뮬레이션)
12. AMM과 프론트엔드는 토큰을 어떻게 쓰나
13. 보안 메모, 알려진 한계, 개선점
14. 스스로 점검하기 (문제와 답)
15. 부록: 한 장 요약

이 문서를 읽는 법

각 장은 **개념 → 실제 코드 → 한 줄씩 해설 → 숫자 예시** 순서입니다. 보라색 상자는 **문법 메모**, 주황 상자는 **주의할 점**, 초록 상자는 **핵심 정리**입니다. 코드 옆에 반드시 숫자를 대입해 보면서 읽으세요.

CHAPTER 1

ERC-20이란: 장부 3개와 함수 6개

ERC-20은 이더리움 계열 체인(BSC, Sepolia 포함)에서 "대체 가능한 토큰"을 만들 때 지키는 **약속(표준)**입니다. "대체 가능"이란 내 1 SW42와 남의 1 SW42가 완전히 같다는 뜻입니다. (NFT는 번호마다 다르므로 대체 불가능입니다.)

모든 지갑, 거래소, AMM이 같은 함수 이름으로 토큰을 다룰 수 있는 이유가 바로 이 표준 덕분입니다. MetaMask가 처음 보는 토큰의 잔액을 보여줄 수 있는 것도 `balanceOf` 라는 이름이 약속되어 있기 때문입니다.

1-1. 토큰 컨트랙트는 "장부"입니다

토큰은 지갑 안에 들어 있는 게 아닙니다. **토큰 컨트랙트 안의 장부**에 "누가 얼마"라고 적혀 있을 뿐입니다. 지갑(MetaMask)은 그 장부를 읽어서 보여주는 창입니다.

장부 (OpenZeppelin 변수명)	모양	뜻
`_balances`	`주소 → 숫자`	누가 몇 개를 가졌나
`_allowances`	`주인 → (사용자 → 숫자)`	주인이 누구에게 몇 개까지 써도 된다고 허락했나
`_totalSupply`	`숫자 하나`	지금까지 존재하는 토큰 총량

핵심 불변식

모든 주소의 `_balances` 를 더하면 언제나 `_totalSupply` 와 같습니다. 토큰이 새로 생기면(mint) 둘 다 늘고, 없어지면(burn) 둘 다 줄고, 옮기면(transfer) 합계는 그대로입니다.

1-2. 표준 함수 6개 + 이벤트 2개

함수	종류	하는 일
`totalSupply()`	읽기	총발행량
`balanceOf(account)`	읽기	account의 잔액
`transfer(to, value)`	쓰기	**내** 토큰을 to에게 보냄
`approve(spender, value)`	쓰기	spender가 내 토큰을 value까지 쓰도록 허락
`allowance(owner, spender)`	읽기	owner가 spender에게 남겨 둔 허락량
`transferFrom(from, to, value)`	쓰기	허락받은 사람이 from의 토큰을 to에게 보냄

이벤트	언제
`Transfer(from, to, value)`	잔액이 움직일 때마다 (mint면 from = 0x0, burn이면 to = 0x0)
`Approval(owner, spender, value)`	approve로 허락량을 정할 때

여기에 이름표 정보 3개(`name()`, `symbol()`, `decimals()`)가 더해집니다. 이것은 `IERC20Metadata`라는 확장 표준입니다.

1-3. 우리 프로젝트의 토큰 두 개

	Token A	Token B
파일	`SunWoo42.sol`	`tokenB.sol`
컨트랙트 이름	`SunWoo42`	`TokenB`
`name()`	SunWoo42	Ra2techB
`symbol()`	SW42	R2B
`decimals()`	18	18
상속	`ERC20, ERC20Permit, Ownable` (같음)	
직접 만든 기능	`faucet()`, `mint(to, amount)`, `lastFaucetClaim`, `FaucetClaimed` (같음)	
쿨다운 표기	`86400`	`1 days` (값은 같음)
faucet 조건식	`now >= last + COOLDOWN`	`now - last >= COOLDOWN`

두 파일은 거의 같고, 세부 표기만 조금 다릅니다. 이 차이는 8장에서 비교합니다. 우리가 직접 쓴 코드는 30줄 정도이고, 나머지 ERC-20 기능은 전부 OpenZeppelin에서 **상속**받습니다.

CHAPTER 2

단위: wei, decimals, 10^{18}

블록체인에는 **소수점이 없습니다**. Solidity의 숫자 타입은 정수뿐입니다(`uint256`). 그런데 "1.5 SW42"처럼 소수점 금액은 필요합니다. 그래서 **아주 작은 단위의 정수**로 저장하고, 화면에 보여줄 때만 소수점을 찍습니다.

2-1. decimals = 18의 의미

`decimals()`가 18이라는 것은 "장부의 숫자를 10^{18}으로 나눈 값이 사람이 보는 토큰 개수"라는 약속입니다.

사람이 보는 값	장부에 저장되는 정수 (wei 단위)	지수 표기
1 SW42	1,000,000,000,000,000,000	1e18
1.5 SW42	1,500,000,000,000,000,000	1.5e18
0.1 SW42	100,000,000,000,000,000	1e17
1000 SW42 (faucet)	1,000,000,000,000,000,000,000	1e21
1,000,000 SW42 (초기 발행)	1,000,000,000,000,000,000,000,000	1e24
가장 작은 단위	1	0.000000000000000001 SW42

이 가장 작은 단위를 관례상 **wei**라고 부릅니다. 원래는 ETH의 최소 단위 이름인데, 18자리 토큰에서도 같은 말을 씁니다.

> **비유**
>
> 원화를 "원" 대신 "1/10^{18}원"이라는 아주 작은 동전으로만 세는 은행이라고 생각하세요. 장부에는 동전 개수(정수)만 적고, 통장 앱이 화면에 보여줄 때 10^{18}으로 나눠 "1.5원"이라고 표시합니다.

2-2. 코드 속 숫자 읽는 법

```
uint256 public constant FAUCET_AMOUNT = 1_000 * 10 ** 18;
```

조각	뜻
`1_000`	1000. 밑줄(`_`)은 사람이 읽기 쉽게 끊어 쓰는 구분자이고 값에는 영향이 없습니다.
`10 ** 18`	10의 18제곱. `**`는 거듭제곱 연산자입니다.
`1_000 * 10 ** 18`	$1000 \times 10^{18} = 10^{21}$ wei = **1000 토큰**. (`**`가 `*`보다 먼저 계산됩니다.)

문법 메모: 단위 접미사 (unit suffix)

Solidity에는 숫자 뒤에 붙이는 단위가 있습니다. 컴파일할 때 정수로 바뀝니다.

표기	실제 값	어디서 봤나
`1 ether`	10^{18}	테스트·배포 스크립트의 `1_000_000 ether`
`1 gwei`	10^{9}	가스 가격
`1 days`	86400 (초)	`tokenB.sol`의 쿨다운
`1 hours`	3600	

`ether`는 "이더리움 코인"이라는 뜻이 아니라 **×10^{18}**이라는 뜻입니다. 그래서 `1_000_000 ether`는 토큰 100만 개를 wei로 표현한 것입니다(토큰도 decimals 18이니까). `86400`과 `1 days`는 완전히 같은 값입니다.

2-3. uint256은 얼마나 큰가

`uint256`의 최댓값(`type(uint256).max`)은 $2^{256} - 1$입니다.

```
115792089237316195423570985008687907853269984665640564039457584007913129639935   // 약 1.16 × 10^77
```

wei로 쳐도 토큰 1.16 × 10^{59}개까지 표현할 수 있으니, 발행량이 넘칠 걱정은 사실상 없습니다. 이 최댓값은 7장에서 "무제한 허락(infinite approval)"의 표시로 다시 등장합니다.

2-4. 프론트엔드에서는 왜 bigint를 쓰나

JavaScript의 일반 숫자(`number`)는 정확하게 표현할 수 있는 정수가 **$2^{53} - 1$ = 9,007,199,254,740,991**(약 9 × 10^{15})까지입니다.

값	wei	JS number로 정확?
0.009 토큰	9,000,000,000,000,000	겨우 가능
1 토큰	1,000,000,000,000,000,000	❌ 오차 발생

1 토큰조차 정확히 담을 수 없으므로, 프론트엔드는 끝에 `n`이 붙는 **bigint**(`1000000000000000000n`)를 씁니다. viem의 두 함수가 사람 표기와 wei를 오갑니다.

함수	입력	출력
`parseUnits("1.5", 18)`	사람이 입력한 문자열	`1500000000000000000n`
`formatUnits(1500000000000000000n, 18)`	장부의 정수	`"1.5"`

2-5. 계산 연습

문제	답
2.25 SW42는 몇 wei?	2.25×10^{18} = 2,250,000,000,000,000,000
500000000000000000 wei는 몇 개?	$5 \times 10^{17} \div 10^{18}$ = 0.5개
faucet을 3번 받으면 wei로?	3×10^{21} = 3,000,000,000,000,000,000,000
`1_000_000 ether`는?	$10^{6} \times 10^{18} = 10^{24}$ wei = 100만 개
정수 나눗셈 `7 / 2`는?	**3** (Solidity와 bigint 모두 소수점 아래를 버림)

주의: 나눗셈은 항상 버림

Solidity에서 `7 / 2 = 3`입니다. 그래서 금액 계산은 **곱셈을 먼저, 나눗셈을 나중에** 해야 오차가 작습니다. 예: `a * b / c` (좋음) vs `a / c * b` (먼저 버려져서 오차가 커짐).

CHAPTER 3

파일 첫 부분: SPDX, pragma, import, NatSpec

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

// OpenZeppelin ERC20 standard implementation
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
// ERC20Permit adds gasless approval via EIP-2612 signatures
import "@openzeppelin/contracts/token/ERC20/extensions/ERC20Permit.sol";
// Ownable provides access control, allowing only the owner to perform certain actions
import "@openzeppelin/contracts/access/Ownable.sol";
```

3-1. 한 줄씩

줄	뜻
`// SPDX-License-Identifier: MIT`	이 코드의 라이선스. 컴파일러가 없으면 경고를 냅니다. 블록 탐색기에 소스를 공개할 때도 표시됩니다.
`pragma solidity ^0.8.28;`	"0.8.28 이상, 0.9.0 미만 컴파일러로만 컴파일하라." `^`는 같은 메이저(0.8) 안에서 그 이상을 허용한다는 뜻입니다.
`import "@openzeppelin/..."`	다른 파일의 코드를 가져옴. `@openzeppelin/`은 `foundry.toml`의 remapping이 `lib/openzeppelin-contracts/`로 연결해 줍니다.

문법 메모: 0.8 버전이 중요한 이유

Solidity 0.8부터는 **덧셈·뺄셈이 넘치면(overflow / underflow) 자동으로 revert**합니다. 예전(0.7 이하)에는 `0 - 1`이 조용히 $2^{256} - 1$이 되어 버리는 사고가 많았습니다. 그래서 0.8에서는 `a - b`를 쓸 때 b가 더 크면 tx가 실패합니다. 7장의 `unchecked`는 이 검사를 일부러 끄는 문법입니다.

문법 메모: import 두 가지 모양

`import "경로";`는 그 파일의 이름을 **전부** 가져옵니다(토큰 파일이 쓰는 방식). `import {ERC20} from "경로";`는 **필요한 이름만** 골라 가져옵니다(테스트·스크립트가 쓰는 방식). 골라 가져오는 쪽이 이름 충돌이 없고 어디서 왔는지 분명해서 요즘 권장되는 스타일입니다.

3-2. NatSpec 주석

```
/**
 * @title SunWoo42
 * @notice Test ERC-20 token used as Token A of the TokenZswap AMM pool.
 * ...
 */
```

`/** ... */`와 `///`는 **NatSpec**이라는 문서용 주석입니다. 컴파일러가 이 주석을 읽어 문서 데이터를 만듭니다.

태그	용도
`@title`	컨트랙트 이름표
`@notice`	사용자에게 보여줄 설명 (무엇을 하는가)
`@dev`	개발자용 메모 (어떻게, 왜, 주의점)
`@param`	인자 설명. 이름이 실제 인자와 같아야 함
`@return`	반환값 설명

NatSpec 위치 규칙

NatSpec은 설명할 대상 **바로 위**에 둡니다. `SunWoo42.sol`에서는 주석과 `constructor`, `faucet` 사이에 빈 줄이 있습니다. `forge inspect SunWoo42 userdoc`으로 확인해 보니 빈 줄이 있어도 문서는 제대로 붙었습니다. 그래도 읽는 사람이 헷갈리지 않게 붙여 쓰는 것이 정석입니다(`tokenB.sol`처럼).

SunWoo42의 머리 주석은 이 토큰의 **공급 모델**과 **보안 메모**를 정리합니다.

공급 경로	누가	얼마
배포 시 `initialSupply`	배포자 1회	배포 스크립트 기준 100만 개
`faucet()`	누구나, 주소당 하루 1회	1000개
`mint(to, amount)`	owner만	제한 없음

즉 총량이 **고정되지 않은** 토큰입니다. 테스트넷 학습용이라서 의도적으로 이렇게 설계했습니다.

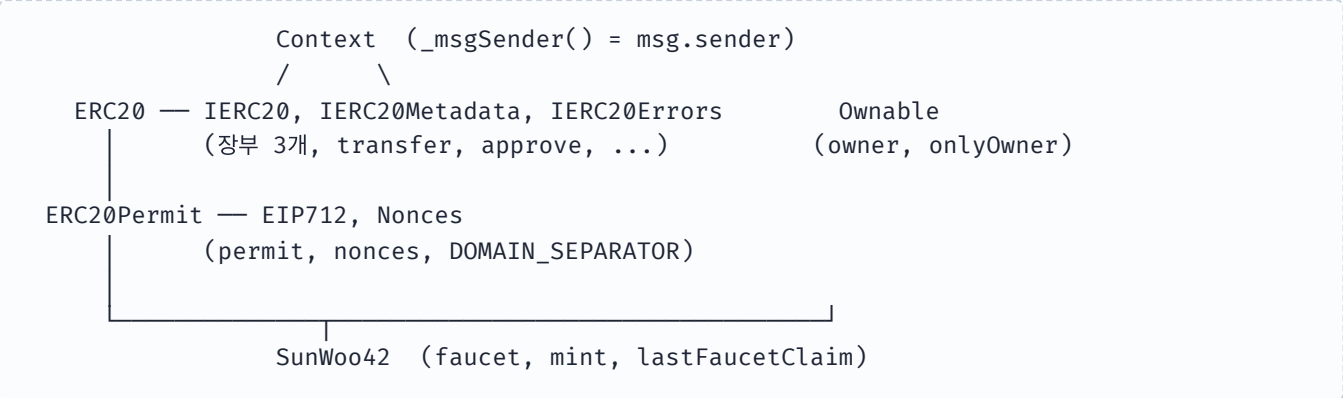
CHAPTER 4

상속: `is ERC20, ERC20Permit, Ownable`

```
contract SunWoo42 is ERC20, ERC20Permit, Ownable {
```

`is`는 "이 컨트랙트들의 코드를 물려받는다"는 뜻입니다. 물려받은 함수와 변수는 내가 쓴 것처럼 존재합니다. 그래서 `SunWoo42.sol`에는 `transfer`나 `balanceOf`가 한 줄도 없지만, 배포된 컨트랙트에는 다 들어 있습니다.

4-1. 무엇을 물려받나



부모	주는 것
`ERC20`	장부 3개, 표준 함수 6개, `name/symbol/decimals`, 내부 함수 `_mint/_burn/_update/_approve/_spendAllowance`
`ERC20Permit`	`permit()` : 서명으로 approve (10장)
`Ownable`	`owner()`, `onlyOwner`, `transferOwnership`, `renounceOwnership` (9장)

문법 메모: 다중 상속 순서

`ERC20Permit`은 이미 `ERC20`을 상속하고 있습니다. 그런데도 `is ERC20, ERC20Permit`처럼 둘 다 적을 수 있는 이유는, Solidity가 **C3 선형화**라는 규칙으로 상속 순서를 한 줄로 정리하기 때문입니다. 같은 부모가 여러 번 나와도 **한 번만** 들어갑니다. 규칙은 "**더 기본적인 것을 왼쪽에, 더 구체적인 것을 오른쪽에**"입니다. `ERC20`(기본) → `ERC20Permit`(ERC20을 확장)의 순서가 맞습니다. 반대로 `is ERC20Permit, ERC20`이라고 쓰면 `Linearization of inheritance graph impossible` 컴파일 에러가 납니다(직접 컴파일해서 확인).

문법 메모: abstract contract

OpenZeppelin의 `ERC20`, `Ownable`은 `abstract contract`입니다. 그 자체로는 배포할 수 없고, **상속받아서 쓰라**는 뜻입니다. 부품은 혼자 팔지 않고 완제품(SunWoo42)에 조립해서 쓰는 셈입니다.

4-2. 왜 직접 짜지 않고 상속하나

- **검증된 코드**: 수많은 프로젝트가 쓰고 감사(audit)를 받은 구현입니다. 직접 짜면 잔액 검사나 0 주소 처리 같은 곳에서 실수하기 쉽습니다.
- **표준 준수**: 이벤트, 반환값, 에러까지 표준과 정확히 맞습니다.
- **우리 코드가 짧아짐**: 우리 관심사(faucet, mint)만 쓰면 됩니다.

CHAPTER 5

상태 변수: constant, mapping, event

```
/// @notice Amount of tokens dispensed by the faucet (in wei units, 18 decimals).
uint256 public constant FAUCET_AMOUNT = 1_000 * 10 ** 18;

/// @notice Cooldown period between faucet claims (in seconds).
uint256 public constant FAUCET_COOLDOWN = 86400;

/// @notice Timestamp (unix seconds) of the last faucet claim per address; 0 if never
claimed.
mapping(address => uint256) public lastFaucetClaim;

/// @notice Emitted when an address receives tokens from the faucet.
event FaucetClaimed(address indexed recipient, uint256 amount);
```

5-1. `uint256 public constant FAUCET_AMOUNT = ...`

조각	뜻
`uint256`	부호 없는 256비트 정수 (0 이상)
`public`	밖에서 읽을 수 있게 **자동 getter 함수**를 만들어 줌 → `FAUCET_AMOUNT()` 호출 가능
`constant`	절대 바뀌지 않는 값. **storage에 저장하지 않고** 코드(바이트코드)에 직접 박힘 → 읽을 때 가스가 거의 들지 않음
`FAUCET_AMOUNT`	상수는 `대문자_밑줄`로 쓰는 관례

문법 메모: constant vs immutable vs 일반 변수

종류	값이 정해지는 때	저장 위치	예
`constant`	컴파일할 때 (코드에 직접 씀)	바이트코드	`FAUCET_AMOUNT`
`immutable`	배포할 때 (constructor에서 한 번)	바이트코드	AMM의 `tokenA`
일반 변수	언제든 바뀜	storage (비쌈)	`lastFaucetClaim`

5-2. `mapping(address => uint256) public lastFaucetClaim;`

mapping은 "키 → 값" 표입니다. 여기서는 "주소 → 마지막으로 faucet을 받은 시각(초)"입니다.

키 (address)	값 (uint256, 유닉스 초)	의미
0xf39F···2266	1,792,399,878	그 시각에 받음
0x8124···9619	0	한 번도 안 받음

- **아무것도 쓰지 않은 키를 읽으면 0**이 나옵니다. "기록 없음 = 0"이라는 성질을 faucet이 그대로 이용합니다.
- `public`이라서 `lastFaucetClaim(주소)`라는 getter가 자동으로 생깁니다. 프론트나 `cast call`에서 바로 조회할 수 있습니다.
- mapping은 "키 목록"을 모릅니다. "지금까지 받은 사람 전체"는 컨트랙트가 알려줄 수 없고, 이벤트를 모아야 합니다(서브그래프의 역할).

> **문법 메모: 이름 붙은 mapping**
>
> OpenZeppelin은 `mapping(address account => uint256) private _balances;`처럼 키에 이름을 붙입니다(0.8.18부터 가능). 동작은 같고, 읽는 사람을 위한 설명입니다. 우리 코드도 `mapping(address user => uint256)`처럼 쓸 수 있습니다.

5-3. `event FaucetClaimed(address indexed recipient, uint256 amount);`

이벤트는 tx 실행 중에 남기는 **로그**입니다. 컨트랙트 자신은 나중에 이 로그를 읽을 수 없습니다. 대신 프론트엔드, 블록 탐색기, 서브그래프가 읽습니다.

부분	뜻
`event FaucetClaimed(...)`	로그의 모양 선언
`emit FaucetClaimed(msg.sender, FAUCET_AMOUNT);`	실제로 로그 남기기 (faucet 함수 안)
`indexed`	이 값을 **topic**에 넣어 검색 가능하게 함. "recipient가 나인 로그만"처럼 필터링 가능. 이벤트당 최대 3개
indexed가 아닌 값	**data** 영역에 들어감. 읽을 수는 있지만 필터 조건으로는 못 씀

로그에 실제로 기록되는 모양:

```
topic0 = keccak256("FaucetClaimed(address,uint256)")   // 이벤트 종류 표시
topic1 = recipient (indexed)
data   = amount = 1000000000000000000000            // 1000 × 10^18
```

> **왜 이벤트가 필요할까**
>
> storage는 "현재 상태"만 보여주고, "언제 누가 받았는지"의 **역사**는 남기지 않습니다. 이벤트는 그 역사를 싸게 남기는 방법입니다. storage에 쓰는 것보다 가스가 훨씬 적게 듭니다.

CHAPTER 6

constructor: 토큰이 태어나는 순간

```
constructor(uint256 initialSupply)
    ERC20("SunWoo42", "SW42")   // token name and ticker symbol
    ERC20Permit("SunWoo42")     // domain separator for EIP-2612
    Ownable(msg.sender)         // sets the deployer as the initial owner
{
    _mint(msg.sender, initialSupply);
}
```

constructor는 배포할 때 **딱 한 번** 실행되는 함수입니다. 배포가 끝나면 다시 호출할 방법이 없습니다.

6-1. 부모 생성자 호출

함수 이름표와 `{` 사이에 있는 `ERC20(...)`, `ERC20Permit(...)`, `Ownable(...)`는 **부모 컨트랙트의 constructor에 인자를 넘기는 문법**입니다.

호출	부모 안에서 일어나는 일
`ERC20("SunWoo42", "SW42")`	`_name = "SunWoo42"`, `_symbol = "SW42"` 저장
`ERC20Permit("SunWoo42")`	`EIP712("SunWoo42", "1")` 호출 → 서명 도메인 이름 설정 (10장)
`Ownable(msg.sender)`	`_owner = 배포자`, `OwnershipTransferred(0x0, 배포자)` 이벤트

주의: 적는 순서 ≠ 실행 순서

부모 생성자는 여기에 **적은 순서가 아니라 상속 선형화 순서**(기본 → 구체)대로 실행됩니다. 대략 Context → ERC20 → EIP712 → Nonces → ERC20Permit → Ownable → 마지막으로 SunWoo42의 본문 `{ _mint(...) }` 순서입니다. 그래서 본문이 실행될 때는 이름, owner 설정이 모두 끝나 있습니다.

문법 메모: TokenB는 한 줄로

`tokenB.sol`은 같은 내용을 한 줄로 씁니다: `constructor(uint256 initialSupply) ERC20("Ra2techB", "R2B") ERC20Permit("Ra2techB") Ownable(msg.sender) { ... }`. 줄바꿈은 문법에 영향이 없습니다. `ERC20Permit`에 넘기는 이름은 `name()`과 같게 맞추는 것이 관례입니다(지갑이 서명 화면에 이 이름을 보여줌).

6-2. 본문: `_mint(msg.sender, initialSupply);`

조각	뜻
`msg.sender`	지금 이 호출을 한 주소. constructor에서는 **배포한 주소**
`initialSupply`	배포할 때 넘긴 인자 (wei 단위)
`_mint`	ERC20의 내부 함수. 새 토큰을 만들어 지정한 주소에 넣음 (7장)

6-3. 숫자로: 배포 스크립트의 경우

```
uint256 constant INITIAL_SUPPLY = 1_000_000 ether;   // Deploy.s.sol
SunWoo42 tokenA = new SunWoo42(INITIAL_SUPPLY);
```

항목	배포 직후 값
`initialSupply`	1,000,000 × 10^{18} = 10^{24} wei
`_balances[배포자]`	10^{24} (100만 SW42)
`_totalSupply`	10^{24}
`_owner`	배포자
이벤트	`OwnershipTransferred(0x0 → 배포자)`, `Transfer(0x0 → 배포자, 10^24)`

자주 하는 실수

`new SunWoo42(1_000_000)`이라고 쓰면 100만 개가 아니라 **100만 wei = 0.000000000001 토큰**이 됩니다. 인자가 wei 단위라는 것을 NatSpec에 적어 둔 이유입니다("pass 1_000_000 * 10**18 for 1,000,000 SW42").

CHAPTER 7

OpenZeppelin ERC20 내부 들여다보기

우리 파일에는 없지만 상속으로 들어와 있는 코드입니다. 위치: `lib/openzeppelin-contracts/contracts/token/ERC20/ERC20.sol` (주석을 뺀 핵심만 옮겼습니다).

7-1. 장부 선언

```
mapping(address account => uint256) private _balances;
mapping(address account => mapping(address spender => uint256)) private _allowances;
uint256 private _totalSupply;
string private _name;
string private _symbol;
```

`private`이라 자식(SunWoo42)도 **직접** 건드릴 수 없습니다. 반드시 `_mint`, `_update` 같은 함수를 거쳐야 합니다. 장부를 아무나 고치지 못하게 하는 안전장치입니다.

`_allowances`는 **2단 mapping**입니다. `_allowances[주인][사용자] = 허락량` 입니다.

7-2. 읽기 함수

```
function name() public view virtual returns (string memory) { return _name; }
function symbol() public view virtual returns (string memory) { return _symbol; }
function decimals() public view virtual returns (uint8) { return 18; }
function totalSupply() public view virtual returns (uint256) { return _totalSupply; }
function balanceOf(address account) public view virtual returns (uint256) { return
_balances[account]; }
function allowance(address owner, address spender) public view virtual returns (uint256)
{
    return _allowances[owner][spender];
}
```

문법 메모: 함수 선언의 단어들

단어	뜻
`public`	밖에서도, 안에서도 호출 가능
`external`	밖에서만 호출 가능 (faucet, mint가 이것)
`internal`	이 컨트랙트와 자식만 호출 가능. 관례상 이름이 `_`로 시작
`view`	상태를 읽기만 함 → 가스 없이 조회 가능
`pure`	상태를 읽지도 않음. `decimals`는 상수만 반환하지만 `view`로 선언되어 있습니다. 자식이 storage를 읽는 버전으로 override할 수 있게 하려는 것입니다
`virtual`	자식이 `override`로 바꿔 쓸 수 있음. 예: decimals를 6으로 바꾸고 싶으면 override
`returns (string memory)`	문자열·배열 같은 참조 타입은 위치(`memory`)를 적어야 함

`decimals()`는 저장된 값이 아니라 **상수 18을 그냥 반환**합니다. 장부에는 wei 정수만 있고, 18은 "어떻게 보여줄지"에 대한 약속일 뿐입니다.

7-3. transfer: 내 토큰 보내기

```
function transfer(address to, uint256 value) public virtual returns (bool) {
    address owner = _msgSender();    // = msg.sender (Context)
    _transfer(owner, to, value);
    return true;
}
```

보내는 사람은 인자가 아니라 `msg.sender`입니다. **남의 토큰을 보낼 방법이 없게** 만든 구조입니다. `_msgSender()`는 `Context`에 정의된 함수로, 그냥 `msg.sender`를 돌려줍니다(메타 트랜잭션 확장을 위한 한 단계 포장).

7-4. approve: 허락량 정하기

```
function approve(address spender, uint256 value) public virtual returns (bool) {
    address owner = _msgSender();
    _approve(owner, spender, value);
    return true;
}
```

owner도 인자가 아니라 `msg.sender`입니다. 그래서 approve는 **반드시 토큰 주인이 직접** 보내야 합니다. AMM이 사용자 대신 approve할 수 없는 이유입니다.

approve는 "더하기"가 아니라 "덮어쓰기"

`approve(AMM, 100)` 후 `approve(AMM, 50)`을 하면 150이 아니라 **50**입니다. `approve(AMM, 0)`은 허락 취소입니다.

7-5. transferFrom: 허락받은 사람이 옮기기

```
function transferFrom(address from, address to, uint256 value) public virtual returns
(bool) {
    address spender = _msgSender();          // 호출한 쪽 = 허락받은 사람 (예: AMM)
    _spendAllowance(from, spender, value);   // ① 허락량 확인·차감
    _transfer(from, to, value);              // ② 실제 이동
    return true;
}
```

함수	owner는 어디서	spender는 어디서
`approve(spender, value)`	`msg.sender`	1번째 인자
`allowance(owner, spender)`	1번째 인자	2번째 인자
`transferFrom(from, to, value)`	`from` 인자	`msg.sender`

같은 장부 칸 `_allowances[owner][spender]`를 세 함수가 서로 다른 방식으로 가리킵니다. "숨어 있는 쪽은 항상 `msg.sender`"라고 기억하세요.

7-6. _spendAllowance: 허락량 차감 (무제한 예외)

```
function _spendAllowance(address owner, address spender, uint256 value) internal virtual
{
    uint256 currentAllowance = allowance(owner, spender);
    if (currentAllowance < type(uint256).max) {             // 무제한이 아니면
        if (currentAllowance < value) {
            revert ERC20InsufficientAllowance(spender, currentAllowance, value);
        }
        unchecked {
            _approve(owner, spender, currentAllowance - value, false);   // 차감, 이벤트 없음
        }
    }
}
```

- 허락량이 `type(uint256).max`(2장의 그 거대한 수)이면 **차감하지 않습니다**. "무제한 허락"의 표시입니다. Foundry로 확인한 결과 `transferFrom` 후에도 max 그대로였습니다.
- 마지막 인자 `false`는 "Approval 이벤트를 내지 마라"입니다. 그래서 transferFrom에서는 **Transfer 이벤트만** 남습니다.

문법 메모: unchecked { }

0.8부터 뺄셈은 자동으로 underflow를 검사합니다(검사에도 가스가 듭니다). 바로 위에서 `currentAllowance >= value`를 이미 확인했으므로 `currentAllowance - value`는 절대 음수가 될 수 없습니다. 그래서 `unchecked`로 중복 검사를 꺼서 가스를 아낍니다. **확실히 안전할 때만** 쓰는 문법입니다.

7-7. _update: 모든 잔액 변화의 단일 통로

```
function _update(address from, address to, uint256 value) internal virtual {
    if (from == address(0)) {
        _totalSupply += value;                          // (A) 발행: 총량 증가
    } else {
        uint256 fromBalance = _balances[from];
        if (fromBalance < value) {
            revert ERC20InsufficientBalance(from, fromBalance, value);
        }
        unchecked { _balances[from] = fromBalance - value; }   // 보내는 쪽 차감
    }

    if (to == address(0)) {
        unchecked { _totalSupply -= value; }            // (B) 소각: 총량 감소
    } else {
        unchecked { _balances[to] += value; }           // 받는 쪽 증가
    }

    emit Transfer(from, to, value);
}
```

`address(0)`(0x000…000)을 "**없음**"의 표시로 씁니다. 그래서 하나의 함수가 세 가지 상황을 모두 처리합니다.

상황	from	to	_totalSupply	잔액	이벤트
발행 (mint)	0x0	받는 사람	+value	to +value	Transfer(0x0 → to)
이동 (transfer)	보내는 사람	받는 사람	그대로	from –, to +	Transfer(from → to)
소각 (burn)	태우는 사람	0x0	–value	from –value	Transfer(from → 0x0)

왜 unchecked가 안전한가

`_balances[to] += value`: 어떤 잔액도 `_totalSupply`보다 클 수 없고, `_totalSupply += value`는 (A)에서 이미 overflow 검사를 거쳤습니다. `_totalSupply -= value`: 태우는 사람의 잔액이 value 이상임을 이미 확인했고, 잔액 ≤ 총량이므로 음수가 될 수 없습니다.

7-8. _transfer, _mint, _burn: _update 앞의 안전 검사

```
function _transfer(address from, address to, uint256 value) internal {
    if (from == address(0)) revert ERC20InvalidSender(address(0));
    if (to == address(0))   revert ERC20InvalidReceiver(address(0));
    _update(from, to, value);
}
function _mint(address account, uint256 value) internal {
    if (account == address(0)) revert ERC20InvalidReceiver(address(0));
    _update(address(0), account, value);
}
function _burn(address account, uint256 value) internal {
    if (account == address(0)) revert ERC20InvalidSender(address(0));
    _update(account, address(0), value);
}
```

0x0으로 `transfer`하는 실수(=사실상 소각)를 막고, mint/burn은 반드시 이 전용 함수로만 하게 합니다. 참고로 우리 AMM의 LP 토큰 회수(`removeLiquidity`)는 이 `_burn`을 씁니다.

7-9. 커스텀 에러 (ERC-6093)

에러	언제
`ERC20InsufficientBalance(sender, balance, needed)`	잔액보다 많이 보내려 할 때
`ERC20InsufficientAllowance(spender, allowance, needed)`	허락량보다 많이 가져가려 할 때 (approve 없이 swap 등)
`ERC20InvalidSender(0x0)` / `ERC20InvalidReceiver(0x0)`	0 주소가 끼어 있을 때
`ERC20InvalidApprover` / `ERC20InvalidSpender`	approve에 0 주소

문법 메모: require 문자열 vs 커스텀 에러

우리 faucet은 `require(조건, "메시지")`를 씁니다. OpenZeppelin은 `if (!조건) revert 에러이름(값들);`을 씁니다. 커스텀 에러는 문자열을 저장하지 않아 **배포·실행 가스가 적고**, 실패 당시의 **숫자를 함께 돌려줍니다** (예: "잔액 300인데 400을 보내려 함"). 디버깅에 훨씬 유리합니다.

CHAPTER 8

faucet(): 누구나 1일 1회 1000개

8-1. SunWoo42의 faucet

```
function faucet() external {
    require(
        block.timestamp >= lastFaucetClaim[msg.sender] + FAUCET_COOLDOWN,
        "Faucet: cooldown period not yet passed"
    );                                                  // ① Checks
    lastFaucetClaim[msg.sender] = block.timestamp;      // ② Effects
    _mint(msg.sender, FAUCET_AMOUNT);                   // ② Effects (장부 변경)
    emit FaucetClaimed(msg.sender, FAUCET_AMOUNT);      // 로그
}
```

줄	설명
`function faucet() external`	인자 없음, 반환값 없음. 밖에서만 호출. `view`가 아님(상태를 바꾸므로) → 트랜잭션 필요, 가스 필요
`block.timestamp`	지금 블록의 시각 (유닉스 초: 1970-01-01부터 흐른 초)
`lastFaucetClaim[msg.sender]`	호출자가 마지막으로 받은 시각. 처음이면 0
`require(조건, "메시지")`	조건이 거짓이면 **tx 전체를 되돌림(revert)**. 그 전에 바뀐 것도 모두 취소, 남은 가스는 환불
`lastFaucetClaim[msg.sender] = block.timestamp`	지금 시각 기록 → 다음 24시간 동안 막힘
`_mint(msg.sender, FAUCET_AMOUNT)`	새로 1000개 발행 → 총량도 1000 증가
`emit FaucetClaimed(...)`	"누가 얼마를 받았다" 로그

8-2. 시각 계산 예시

anvil에서 실제로 기록된 값을 예로 들겠습니다.

시점	block.timestamp	last + 86400	조건	결과
첫 호출	1,792,399,878	0 + 86,400 = 86,400	1,792,399,878 ≥ 86,400	✅ 통과, last = 1,792,399,878
6분 뒤	1,792,400,236	1,792,486,278	1,792,400,236 ≥ 1,792,486,278 ?	❌ revert
정확히 24시간 뒤	1,792,486,278	1,792,486,278	같음 (≥)	✅ 통과

다음 가능 시각 = 마지막 시각 + 86,400초(= 24 × 60 × 60).

> **함정: 첫 호출도 "block.timestamp ≥ 86400"이 필요합니다**
>
> 처음에는 `last = 0`이므로 조건이 `block.timestamp >= 86400`이 됩니다. 실제 체인은 시각이 약 17억 9천만이라 문제없지만, **Foundry 테스트의 기본 시각은 1**입니다. 그래서 테스트에서 `faucet()`을 그냥 부르면 첫 호출부터 실패합니다. Foundry로 확인한 결과: `block.timestamp = 1`에서 SW42와 R2B 모두 실패했고, `vm.warp(86400)` 후에는 둘 다 1000개씩 받았습니다. faucet을 테스트할 때는 먼저 `vm.warp(...)`로 시계를 옮기세요.

8-3. TokenB의 faucet과 비교

```
// TokenB
require(block.timestamp - lastFaucetClaim[msg.sender] >= FAUCET_COOLDOWN, "Faucet
cooldown: 1 day");
```

	SunWoo42	TokenB
식	`now >= last + 86400`	`now - last >= 86400`
수학적으로	같은 조건 (양변에서 last를 옮긴 것)	
위험 요소	`last + 86400` overflow → last는 과거 시각이라 현실적으로 불가능	`now - last` underflow → last ≤ now라서 불가능 (NatSpec에 적어 둠)
쿨다운 표기	`86400`	`1 days`
revert 메시지	"Faucet: cooldown period not yet passed"	"Faucet cooldown: 1 day"

둘 다 올바르게 동작합니다. 다만 같은 기능인데 표기와 메시지가 달라서, 프론트에서 에러를 처리하거나 문서를 쓸 때 헷갈립니다. 13장의 개선점에 넣어 두었습니다.

8-4. 순서가 왜 이런가: CEI 패턴

Checks → Effects → Interactions: 먼저 검사하고, 그다음 내 장부를 바꾸고, 외부 호출은 마지막에 합니다. faucet에는 외부 컨트랙트 호출이 없지만, **기록(`lastFaucetClaim`)을 발행보다 먼저** 하는 습관이 중요합니다. 만약 발행 과정에서 다른 코드가 끼어들 수 있는 구조였다면, 기록 전에 다시 들어와 두 번 받는 공격(재진입)이 가능했을 것입니다.

8-5. 한 번 호출에 남는 로그 2개

#	이벤트	누가 냄	값
0	`Transfer(0x0 → 호출자, 1000e18)`	OpenZeppelin `_update`	표준 이벤트: 지갑·탐색기가 이걸로 잔액 변화를 인식
1	`FaucetClaimed(호출자, 1000e18)`	우리 코드	"faucet으로 받았다"는 의미를 구분하는 용도

8-6. 프론트에서 실패하면 어디서 막히나

쿨다운 중에 버튼을 누르면 MetaMask가 보내기 전에 `eth_estimateGas`로 미리 실행해 보고 revert를 감지합니다. 그래서 **체인에 기록되지 않고, 가스비도 나가지 않습니다**. 화면에는 revert 메시지가 에러로 표시됩니다.

CHAPTER 9

mint()와 onlyOwner, Ownable

```
function mint(address to, uint256 amount) external onlyOwner {
    _mint(to, amount);
}
```

owner가 원하는 주소(`to`)에 원하는 양(`amount`, wei 단위)을 새로 발행합니다. 상한이 없습니다.

9-1. modifier: onlyOwner는 어떻게 동작하나

```
// Ownable.sol
modifier onlyOwner() {
    _checkOwner();
    _;                      // ← 여기에 원래 함수 본문이 들어감
}
function _checkOwner() internal view virtual {
    if (owner() != _msgSender()) {
        revert OwnableUnauthorizedAccount(_msgSender());
    }
}
```

문법 메모: modifier와 `_;`

modifier는 함수 앞뒤에 끼워 넣는 코드 조각입니다. `_;` 자리에 원래 함수 본문이 들어간다고 생각하면 됩니다. 그래서 `mint`는 실제로 이렇게 실행됩니다.

```
_checkOwner();          // owner가 아니면 여기서 revert
_mint(to, amount);      // 원래 본문
```

9-2. 숫자 예시

호출자	호출	결과
owner (배포자)	`mint(b, 500e18)`	✅ b +500, 총량 +500
b (owner 아님)	`mint(b, 500e18)`	❌ `OwnableUnauthorizedAccount(b)`

9-3. Ownable이 주는 나머지 함수

함수	하는 일
`owner()`	현재 owner 주소 조회. 배포 스크립트가 `tokenA.owner()`로 배포자 주소를 얻을 때 사용
`transferOwnership(newOwner)`	owner만 호출. 소유권 이전 (0 주소는 거부)
`renounceOwnership()`	owner만 호출. owner를 0x0으로 → 이후 **아무도 mint할 수 없음** (되돌릴 수 없음)

중앙화 지점

owner 키를 가진 사람은 토큰을 무한히 찍어낼 수 있습니다. 찍어서 AMM에 팔면 풀의 상대 토큰을 거의 다 가져갈 수 있습니다. 즉 **owner 키 = 이 토큰의 가격 결정권**입니다. 테스트넷 학습용이라 허용했지만, 실제 서비스라면 발행 상한(cap), 멀티시그, 또는 renounce가 필요합니다. 그래서 BSC 배포는 개인키 문자열 대신 **keystore(`--account deployer`)**로 하기로 했습니다.

CHAPTER 10

ERC20Permit: 서명으로 하는 approve

보통 approve는 **트랜잭션**이라 가스가 들고, swap 전에 팝업이 한 번 더 뜹니다. **permit(EIP-2612)**은 "AMM에게 100개 허락함"이라는 내용에 **지갑으로 서명만** 해서(가스 없음, 오프체인) 넘기는 방식입니다. 그 서명을 받은 컨트랙트가 `permit(...)`을 호출하면 그 자리에서 `_approve`가 실행됩니다.

```
function permit(address owner, address spender, uint256 value, uint256 deadline,
                uint8 v, bytes32 r, bytes32 s) public virtual {
    // ① 유효기간 확인
    if (block.timestamp > deadline) revert ERC2612ExpiredSignature(deadline);
    // ② 서명한 내용 + 일회용 번호(nonce)를 해시
    bytes32 structHash = keccak256(abi.encode(PERMIT_TYPEHASH, owner, spender, value,
                                              _useNonce(owner), deadline));
    // ③ 도메인(토큰 이름, 버전, 체인, 주소)을 붙임 (EIP-712)
    bytes32 hash = _hashTypedDataV4(structHash);
    // ④ 서명에서 서명자 주소 복원
    address signer = ECDSA.recover(hash, v, r, s);
    // ⑤ 서명자가 owner 본인인지 확인
    if (signer != owner) revert ERC2612InvalidSigner(signer, owner);
    // ⑥ approve와 똑같이 허락 기록
    _approve(owner, spender, value);
}
```

요소	역할
`deadline`	서명의 유효기간. 지나면 못 씀
`nonce` (`nonces(owner)`)	주소별 일회용 번호. 쓰면 1 증가 → 같은 서명을 두 번 쓰는 재사용 공격 방지
도메인 (`EIP712("SunWoo42", "1")`)	토큰 이름, 버전, 체인 ID, 컨트랙트 주소를 서명에 섞음 → 다른 토큰이나 다른 체인에서 같은 서명을 재사용할 수 없음
`v, r, s`	서명 값 (65바이트를 세 조각으로 나눈 것)

6장에서 `ERC20Permit("SunWoo42")`로 넘긴 이름이 바로 이 **도메인 이름**입니다. MetaMask 서명 화면에도 이 이름이 표시됩니다.

우리 프로젝트에서는 쓰지 않습니다

토큰에는 permit 기능이 있지만, AMM의 `swap` / `addLiquidity`는 서명을 받는 버전이 없습니다. 그래서 프론트는 approve 트랜잭션을 따로 보냅니다(팝업 2번). AMM에 `swapWithPermit(...)` 같은 함수를 추가하면 팝업을 1번으로 줄일 수 있습니다. README의 "개선점"에 적을 만한 항목입니다.

CHAPTER 11

토큰의 일생: 장부 변화 추적

배포부터 시작해서 SW42의 장부가 어떻게 바뀌는지 한 단계씩 따라갑니다. 단위는 토큰 개수(= wei ÷ 10^{18})로 적었습니다. 등장인물: **D** = 배포자(owner), **AMM**, **b**, **c** = 일반 사용자.

#	호출 (msg.sender)	D	AMM	b	c	총량	allowance[D][AMM]
0	`new SunWoo42(10^24)` (D)	1,000,000	0	0	0	1,000,000	0
1	`approve(AMM, 10000e18)` (D)	1,000,000	0	0	0	1,000,000	**10,000**
2	`transferFrom(D, AMM, 10000e18)` (AMM, addLiquidity 안에서)	**990,000**	**10,000**	0	0	1,000,000	**0**
3	`faucet()` (b)	990,000	10,000	**1,000**	0	**1,001,000**	0
4	`faucet()` 1시간 뒤 (b)	❌ revert "Faucet: cooldown period not yet passed" (아무것도 안 바뀜)					
5	`mint(b, 500e18)` (D)	990,000	10,000	**1,500**	0	**1,001,500**	0
6	`mint(b, 500e18)` (b)	❌ revert `OwnableUnauthorizedAccount(b)`					
7	`transfer(c, 300e18)` (b)	990,000	10,000	**1,200**	**300**	1,001,500	0
8	`transfer(b, 400e18)` (c)	❌ revert `ERC20InsufficientBalance(c, 300e18, 400e18)`					

단계별 검산 (불변식: 잔액 합계 = 총량)

#	잔액 합계	총량	일치?
2	990,000 + 10,000 = 1,000,000	1,000,000	✅
3	990,000 + 10,000 + 1,000 = 1,001,000	1,001,000	✅
5	990,000 + 10,000 + 1,500 = 1,001,500	1,001,500	✅
7	990,000 + 10,000 + 1,200 + 300 = 1,001,500	1,001,500	✅ (transfer는 총량 불변)

각 단계에서 나오는 이벤트

#	이벤트
0	`OwnershipTransferred(0x0, D)`, `Transfer(0x0, D, 1,000,000)`
1	`Approval(D, AMM, 10,000)`
2	`Transfer(D, AMM, 10,000)` (Approval은 안 나옴: `_approve(..., false)`)
3	`Transfer(0x0, b, 1,000)`, `FaucetClaimed(b, 1,000)`
5	`Transfer(0x0, b, 500)`
7	`Transfer(b, c, 300)`
4, 6, 8	없음 (revert된 tx는 로그도 함께 사라짐)

정리

revert는 "부분 실패"가 없습니다. 실패한 tx의 장부 변경과 이벤트는 **전부** 없었던 일이 됩니다. 그래서 2번처럼 AMM 안에서 일어나는 `transferFrom`이 실패하면 addLiquidity 전체가 취소되고, 풀 장부도 바뀌지 않습니다.

CHAPTER 12

AMM과 프론트엔드는 토큰을 어떻게 쓰나

12-1. AMM 쪽: approve + transferFrom 패턴

ERC-20은 "토큰이 들어왔다"는 사실을 받는 컨트랙트에 알려주지 않습니다. 그래서 AMM은 다음 패턴을 씁니다.

```
사용자: tokenA.approve(AMM, 100)        ← tx 1: "AMM아, 내 A를 100까지 가져가도 돼"
사용자: amm.swap(tokenA, 100, minOut)   ← tx 2
          └ AMM 안에서: tokenA.safeTransferFrom(사용자 → AMM, 100)   ← AMM이 직접 가져감
                        ... 계산 ...
                        tokenB.safeTransfer(AMM → 사용자, out)
```

AMM이 함수 **안에서** 직접 가져가기 때문에 "토큰 받기 + 계산 + 내보내기"가 한 tx로 묶입니다. 중간에 하나라도 실패하면 전부 되돌아갑니다.

safeTransferFrom은 왜 쓰나

AMM은 `tokenA.transferFrom(...)` 대신 OpenZeppelin `SafeERC20`의 `safeTransferFrom`을 씁니다. 어떤 토큰(대표적으로 예전 USDT)은 표준과 달리 실패해도 revert하지 않고 `false`를 돌려주거나, 아예 반환값이 없습니다. `safe` 버전은 이런 경우까지 실패로 처리해 줍니다. 우리 토큰은 OpenZeppelin이라 항상 표준대로 동작하지만, AMM이 "어떤 토큰이든" 안전하게 다루도록 쓰는 습관입니다.

12-2. 프론트엔드 쪽: tokenAbi

```
export const tokenAbi = parseAbi([
  'function balanceOf(address owner) view returns (uint256)',
  'function faucet()',
  'function approve(address spender, uint256 amount) returns (bool)',
  'function allowance(address owner, address spender) view returns (uint256)',
])
```

함수	프론트에서 쓰는 곳	종류
`balanceOf`	내 잔액 표시. AMM 주소에 물으면 **LP 잔액**(AMM도 ERC-20이라서)	읽기 (팝업 없음)
`faucet`	"A 1000개 받기" 버튼	쓰기 (팝업)
`allowance`	approve가 필요한지 판단: `allowance < amountIn`이면 Approve 버튼	읽기
`approve`	Approve 버튼 (Swap, AddLiquidity)	쓰기

ABI에는 무엇이 들어가나

ABI 한 줄은 Solidity 선언에서 **modifier와 본문을 지운 껍데기**입니다. `external`, `onlyOwner`, `nonReentrant`는 호출하는 쪽과 무관하므로 적지 않습니다. `view` / `pure`는 적어야 합니다. wagmi가 이 값으로 "읽기 함수인지 쓰기 함수인지"를 구분하기 때문입니다. `faucet`을 `view`로 적으면 쓰기 훅에서 쓸 수 없습니다.

12-3. 함수 선택자

EVM은 함수 이름이 아니라 **4바이트 선택자**(= `keccak256("이름(타입들)")`의 앞 4바이트)로 함수를 찾습니다.

시그니처	선택자
`faucet()`	`0xde5f72fd`
`balanceOf(address)`	`0x70a08231`
`approve(address,uint256)`	`0x095ea7b3`
`allowance(address,address)`	`0xdd62ed3e`
`transfer(address,uint256)`	`0xa9059cbb`
`transferFrom(address,address,uint256)`	`0x23b872dd`

직접 계산해 보기: `cast sig "faucet()"`. 인자 타입이 하나라도 다르면 완전히 다른 선택자가 됩니다(`mint()` ≠ `mint(address)`).

12-4. 터미널에서 직접 확인하기

```
# 잔액 (wei로 나옴)
cast call <tokenA> "balanceOf(address)(uint256)" <주소> --rpc-url http://127.0.0.1:8545
# 허락량
cast call <tokenA> "allowance(address,address)(uint256)" <주인> <AMM> --rpc-url
http://127.0.0.1:8545
# 마지막 faucet 시각
cast call <tokenA> "lastFaucetClaim(address)(uint256)" <주소> --rpc-url
http://127.0.0.1:8545
# faucet을 보내지 않고 미리 실행만 해 보기 (상태는 안 바뀜)
cast call <tokenA> "faucet()" --from <주소> --rpc-url http://127.0.0.1:8545
# wei → 사람 단위
cast from-wei 10000000000000000000      # → 1000.000000000000000000
```

CHAPTER 13

보안 메모, 알려진 한계, 개선점

13-1. 알려진 한계 (의도한 것)

항목	내용	왜 허용했나
무제한 mint	owner가 원하는 만큼 발행 가능 → 가격 조작 가능	테스트넷 학습용, 평가용 지갑 충전 필요
faucet 우회	새 지갑을 만들면 쿨다운 무시 (시빌 공격)	토큰에 실제 가치가 없음
`block.timestamp`	검증자가 몇 초 정도 조정 가능	24시간 쿨다운에는 무의미한 오차

13-2. ERC-20 자체의 주의점

approve 경쟁 상태 (race condition)

Alice가 Bob에게 100을 허락했다가 50으로 줄이려고 `approve(Bob, 50)`을 보냈다고 합시다. Bob이 이 tx를 보고, 먼저 처리되도록 `transferFrom 100`을 앞질러 보내면 이렇게 됩니다.

순서	일	Bob이 가져간 양
1	Bob의 `transferFrom(Alice, Bob, 100)` (먼저 처리)	100
2	Alice의 `approve(Bob, 50)` → 허락량 50으로 **덮어씀**	
3	Bob의 `transferFrom(Alice, Bob, 50)`	+50 = **150**

대책: 바꿀 때 먼저 0으로 만든 뒤 새 값을 설정하거나, 우리 프론트처럼 **쓸 만큼만 정확히** approve합니다(swap 한 번에 소진되므로 남는 허락이 없음).

무제한 approve의 위험

`approve(AMM, type(uint256).max)`는 편하지만(7-6장: 줄어들지도 않음), AMM에 버그가 있거나 해킹되면 **지갑의 그 토큰 전부**가 위험해집니다. 우리 프론트는 매번 정확한 양만 approve합니다.

13-3. 개선할 점 (BSC 배포 전에 고치면 좋은 것)

개선점	지금	제안
revert 메시지 통일	SW42 "Faucet: cooldown period not yet passed" / R2B "Faucet cooldown: 1 day"	같은 문구 또는 같은 커스텀 에러 `FaucetCooldown(uint256 nextClaimAt)`
쿨다운 표기 통일	`86400` / `1 days`	둘 다 `1 days` (읽기 쉬움)
조건식 통일	`now >= last + CD` / `now - last >= CD`	한 가지로
NatSpec 위치	SunWoo42는 주석과 함수 사이에 빈 줄 (동작에는 문제 없음)	붙여 쓰기
코드 중복	두 파일이 거의 같음	공통 부모 `FaucetToken`을 만들고 두 토큰이 상속 (이름·심볼만 다르게)
(선택) 발행 상한	없음	`ERC20Capped` 상속

배포 후에는 못 고칩니다

컨트랙트는 배포되면 코드를 바꿀 수 없습니다(업그레이드 패턴을 쓰지 않는 한). 고치려면 **새로 배포**해야 하고, 그러면 주소가 바뀌어 프론트의 `contracts.ts`, 서브그래프 설정, 기존 유동성을 모두 다시 맞춰야 합니다. 그래서 BSC 테스트넷 배포 전에 정리하는 것이 좋습니다.

CHAPTER 14

스스로 점검하기

먼저 답을 가리고 풀어 보세요. 막히면 괄호 안의 장을 다시 읽으세요.

Q1. `FAUCET_AMOUNT`를 wei로 쓰면? (2장)

$1000 \times 10^{18} = 10^{21} = 1{,}000{,}000{,}000{,}000{,}000{,}000$

Q2. `new SunWoo42(1_000_000)`으로 배포하면 배포자는 몇 개를 받나? (6장)

10^{6} wei = $10^{6} \div 10^{18} = 0.000000000001$개. `1_000_000 ether`라고 써야 100만 개입니다.

Q3. `approve(AMM, 100e18)` 직후 내 잔액은? (7장)

그대로입니다. approve는 허락만 기록하고 토큰을 옮기지 않습니다.

Q4. `approve(AMM, 100e18)` 후 `approve(AMM, 30e18)`을 하면 허락량은? (7장)

30. 더하기가 아니라 덮어쓰기입니다.

Q5. AMM이 사용자 대신 `tokenA.approve(...)`를 호출하면 무엇이 허락되나? (7장)

"AMM의 토큰을 누군가에게" 허락하는 것이 됩니다. approve의 owner는 항상 `msg.sender`이기 때문입니다.

Q6. 허락량이 `type(uint256).max`일 때 transferFrom 100을 하면 허락량은? (7-6장)

그대로 max입니다. 무제한 표시라서 차감하지 않습니다.

Q7. faucet에서 처음 받는 사람의 `lastFaucetClaim` 값은? 왜? (5장)

0. mapping에서 한 번도 쓰지 않은 키는 0을 돌려줍니다.

Q8. Foundry 테스트에서 `faucet()`을 바로 부르면 실패하는 이유는? (8장)

기본 `block.timestamp`가 1이라 `1 >= 0 + 86400`이 거짓입니다. `vm.warp(86400)`이 필요합니다.

Q9. faucet 한 번에 로그는 몇 개, 무엇인가? (8장)

2개. `Transfer(0x0 → 나, 1000e18)`(OpenZeppelin)와 `FaucetClaimed(나, 1000e18)`(우리 코드).

Q10. 마지막 수령 시각이 1,792,399,878이면 다음 수령 가능 시각은? (8장)

1,792,399,878 + 86,400 = 1,792,486,278

Q11. transfer 후 `totalSupply`는? mint 후에는? (7-7장)

transfer: 그대로. mint: 발행한 만큼 증가.

Q12. `unchecked`를 써도 되는 조건은? (7장)

바로 앞에서 그 계산이 넘치지 않음을 이미 확인했을 때만입니다(예: `currentAllowance >= value`를 검사한 뒤의 뺄셈).

Q13. ABI에 `'function faucet() view'`라고 쓰면 무엇이 문제인가? (12장)

faucet은 상태를 바꾸는 함수인데 읽기 함수로 분류되어, wagmi의 쓰기 훅(`useWriteContract`)에서 쓸 수 없습니다.

Q14. owner 키가 유출되면 어떤 일이 가능한가? (9장)

토큰을 무한히 발행해서 AMM에 팔아 풀의 상대 토큰을 거의 전부 가져갈 수 있습니다. 소유권 이전도 가능합니다.

Q15. permit의 nonce는 무엇을 막나? (10장)

같은 서명을 두 번 쓰는 재사용(replay)을 막습니다. 쓸 때마다 1씩 증가합니다.

APPENDIX

한 장 요약

주제	한 줄 요약
토큰의 정체	토큰 컨트랙트 안의 장부 3개: `_balances`, `_allowances`, `_totalSupply`
단위	장부는 wei 정수, 사람 단위 = wei ÷ 10^{18}, `1 ether` = 10^{18}, `1 days` = 86400
상속	`ERC20`(장부·표준), `ERC20Permit`(서명 approve), `Ownable`(owner 권한)
constructor	이름·심볼·도메인·owner 설정 후 배포자에게 `initialSupply` 발행
모든 잔액 변화	`_update(from, to, value)` 하나로. from=0이면 발행, to=0이면 소각
transfer	보내는 사람 = `msg.sender`
approve	owner = `msg.sender`, 덮어쓰기, 토큰은 안 움직임
transferFrom	spender = `msg.sender`, 허락량 차감 후 이동 (max면 차감 안 함)
faucet	주소당 24시간에 1번 1000개, CEI 순서, 로그 2개
mint	`onlyOwner`, 상한 없음 = 중앙화 지점
에러	OpenZeppelin은 커스텀 에러(값 포함), 우리 faucet은 require 문자열
프론트	bigint + `parseUnits` / `formatUnits`, allowance가 부족하면 approve 먼저
AMM	approve(tx 1) → AMM이 함수 안에서 `safeTransferFrom`(tx 2)

파일 위치

무엇	경로
Token A	`code/src/SunWoo42.sol`
Token B	`code/src/tokenB.sol`
OpenZeppelin ERC20	`code/lib/openzeppelin-contracts/contracts/token/ERC20/ERC20.sol`
ERC20Permit	`.../token/ERC20/extensions/ERC20Permit.sol`
Ownable	`.../access/Ownable.sol`
배포 스크립트	`code/script/Deploy.s.sol`
프론트 ABI	`frontend/src/contracts.ts` (`tokenAbi`)
Swap 트레이스 문서	`documentation/swap-trace.md`